sequence covers the square more evenly, which is what we'd like to achieve with Thompson sampling. This contrast is even more glaring in higher dimensions, giving us more reason to prefer Sobol sequences than uniformly sampled data points. We won't be discussing Sobol sequences in detail here; what we need to know is that a Sobol sequence is the go-to method of sample generation if our goal is to cover a space evenly.

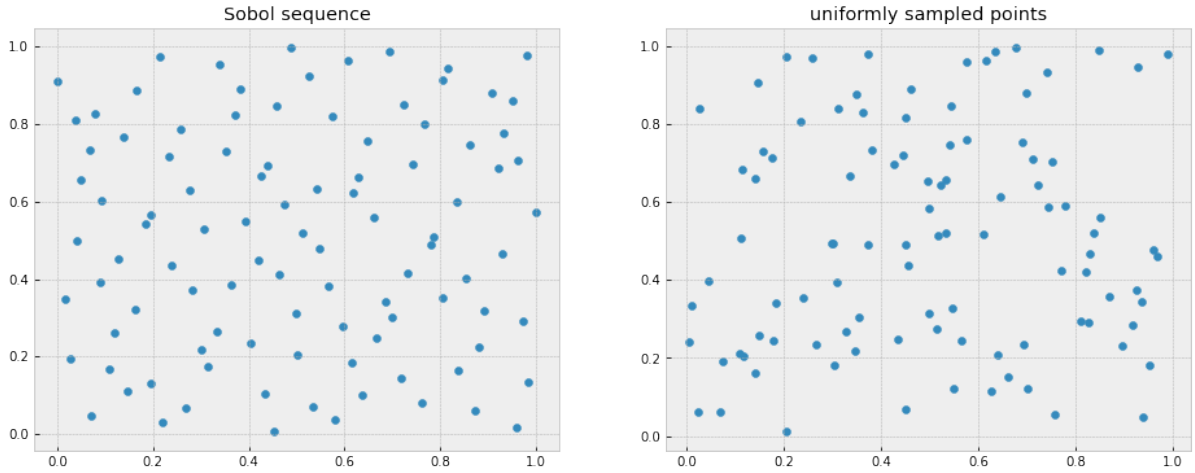


Figure 5.14 Points from a Sobol sequence vs. uniformly sampled points in the two-dimensional unit square. The Sobol sequence covers the square more evenly and therefore should be used by Thompson sampling.

PyTorch provides an implementation of Sobol sequences, which can be used as follows:

```
num_candidates = 1000

sobol = torch.quasirandom.SobolEngine(dim, scramble=True)    ❶
candidate_x = sobol.draw(num_candidates)    ❷
```

❶ `dim` is the number of dimensions of the space.

❷ `num_candidates` is the number of points to be generated.

Here, `sobol` is an instance of the `SobolEngine` class, which implements the sampling logic of Sobol sequences in the unit cube, and `candidate_x` is a PyTorch tensor of the shape `(num_candidates, dim)`, containing the generated points with the correct dimensionality.

NOTE **Resizing the Sobol samples to the correct space**

It's important to remember that `SobolEngine` generates points that cover the unit cube. To get `candidate_x` to cover the space that we want, we will need to resize this tensor accordingly.

How many points the Sobol sequence should contain (that is, the value of `num_candidates`) is up to us the user to decide; the example above shows that we are using one thousand. In a typical case, you would want a value large enough so that the search space could be sufficiently covered.

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>